



Univerzitet u Sarajevu
Elektrotehnički fakultet
Sarajevo

UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET

P16: BVH RAY TRACER

Istraživački izvještaj

CHECKPOINT 1

Predmet: Paralelni računarski sistemi
Mentor: Tarik Hrnjić, MA

Autori

Dino Burić - 2502/2026
Ali Fazlić - 2590/2026
Zijad Doglod - 2576/2026
Ajdin Dželo - 2527/19540

Sarajevo, novembar 2026.

Sažetak

U okviru projekta P16: BVH Ray Tracer istražujemo kako se praćenje zraka (ray tracing) može ubrzati kombinovanjem prostorne strukture Bounding Volume Hierarchy (BVH) i paralelnog izvršavanja. Pregledom literature nastojali smo razumjeti na koji način se BVH stablo gradi, zašto njegov kvalitet utiče na brzinu renderovanja te koje mogućnosti nude višejezgreni procesori i SIMD instrukcije. Naša ideja je izraditi jednostavan ray tracer u C++-u i porediti sekvencijalnu izvedbu bez BVH-a, sekvencijalnu izvedbu s BVH-om i paralelnu verziju s OpenMP-om. Ovaj dokument predstavlja prvu istraživačku fazu (Checkpoint 1); implementacija i mjerenja tek slijede.

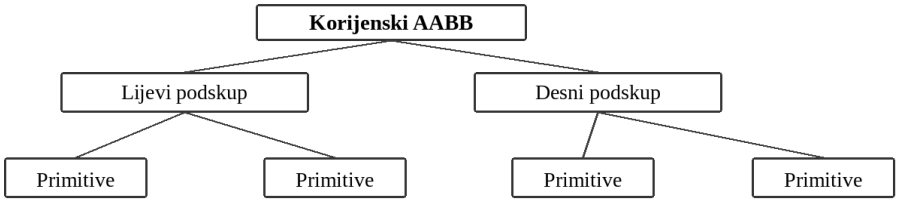
1. Uvod u temu

Ray tracing je postupak generisanja slike u kojem se iz virtualne kamere kroz piksele šalju zrake u trodimenzionalnu scenu. Program pronalazi najbližu površinu koju je pojedina zraka pogodila i na osnovu materijala, osvjetljenja i položaja izvora svjetlosti određuje boju piksela. Dodatne zrake omogućavaju računanje sjena i refleksija. Ovakav način renderovanja može dati uvjerljive rezultate, ali pri većem broju objekata zahtijeva mnogo geometrijskih proračuna [6, 7].

Najjednostavniji, takozvani naivni pristup, svaku zraku provjerava protiv svakog objekta u sceni. Ako scena sadrži N geometrijskih primitiva, broj testova po zraci u najgorem slučaju raste proporcionalno s N. Zbog toga se u praktičnim rendererima koriste akceleracijske strukture koje izbjegavaju nepotrebne provjere [4].

2. BVH i njegova uloga u ray tracingu

BVH (Bounding Volume Hierarchy) je hijerarhijsko stablo koje objekte organizuje u prostorne grupe. Svaki čvor obuhvata određene primitive pomoću graničnog volumena, najčešće AABB kutije (Axis-Aligned Bounding Box), čije su stranice paralelne koordinatnim osama. Korijenski čvor predstavlja cijelu scenu, unutrašnji čvorovi manje grupe, a listovi konkretne objekte. Ako zraka promaši AABB nekog čvora, mogu se preskočiti svi objekti u njegovom podstablu. Tako se umjesto velikog broja skupih presjeka s trouglovima izvodi manji broj jednostavnijih testova sa kutijama [7].



Slika 1. Pojednostavljen prikaz binarnog BVH stabla.

BVH nije zaseban algoritam za osvjetljenje, nego način da ray tracer brže pronađe relevantnu geometriju. Iz literature se vidi da su važne dvije različite stvari: trošak izgradnje stabla i brzina njegovog obilaska tokom renderovanja. Dobro stablo može zahtijevati više vremena za pripremu, ali kasnije smanjiti broj testova presjeka. Zato u projektu planiramo ove dvije faze mjeriti odvojeno [1, 3].

3. Pregled naučnih radova

3.1. Waldemarson i Doggett (2025): Parallel Axis Split Tasks for Bounding Volume Construction with OpenMP [1]

Ovaj rad istražuje kako ubrzati konstrukciju BVH stabla na višejezgrenom CPU-u. Tokom izgradnje potrebno je stalno dijeliti skupove geometrijskih objekata i odlučivati gdje napraviti podjelu. Ako se takvi koraci izvršavaju samo na jednoj niti, konstrukcija može postati usko grlo, naročito kada se scena mijenja. Autori zato analiziraju OpenMP pristupe parallel for, task i taskloop te ispituju kako raspoređivanje zadataka utiče na iskorištenje procesorskih jezgri.

Na testiranom osmojezgrenom procesoru pojedine varijante postižu približno tri do pet puta bržu konstrukciju BVH-a, uz mali uticaj na kvalitet nastalog stabla. Bitno je naglasiti da se rad primarno bavi CPU izgradnjom BVH-a, dok je GPU renderer korišten za evaluaciju dobivenih struktura. Za naš projekat najkorisnija je ideja odvajanja paralelizacije izgradnje od paralelizacije samog renderovanja. Kao dodatni eksperiment možemo istražiti OpenMP zadatke pri konstrukciji, nakon što uspostavimo osnovnu CPU verziju [1].

3.2. Nishimura (2021): Grid-Induced Bounding Volume Hierarchy for Ray Tracing Dynamic Scenes [3]

Nishimura razmatra problem dinamičnih scena: objekti se mogu pomicati, pojavljivati ili nestajati, a BVH koji je ranije izgrađen više ne odgovara novoj geometriji. Predložena metoda GBVH (Grid-Induced BVH) kombinuje prostornu mrežu i hijerarhijsku strukturu kako bi promjene zahvatile samo relevantne dijelove stabla. Umjesto potpunog ponovnog građenja nakon svake promjene, nastoji se izbjeći nepotreban rad i zadržati efikasno praćenje zraka.

U eksperimentima autori koriste C++, OpenMP i Intel Embree na višejezgrenom CPU-u. Porede trošak izgradnje i ažuriranja sa brzinom renderovanja. Iz ovog rada za P16 preuzimamo razmišljanje o razlici između statičnih i dinamičnih scena, kao i metodologiju odvojenog mjerenja pripreme i pretrage. Naš osnovni projekt planiramo na statičnoj sceni; puna implementacija GBVH-a bila bi preširoka za prvi obim i ostaje moguća nadogradnja [3].

3.3. Meister i Bittner (2017/2018): Parallel Locally-Ordered Clustering for BVH Construction [2]

Rad predstavlja PLOC, algoritam koji BVH gradi odozdo prema gore: pojedinačni objekti grupišu se u klastere, koji se potom spajaju u veće cjeline. Prostorni redoslijed postiže se Morton kodovima, a spajanje se paralelizuje na GPU-u pomoću CUDA-e. Objavljena ubrzanja izgradnje i obilaska odnose se na testne scene autora, a ne nužno na naš projekat.

PLOC je koristan kao primjer drugačije konstrukcije od klasičnog top-down BVH-a. Zbog zahtjevnosti CUDA-e, u P16 će prvenstveno služiti kao istraživačka referenca, a eventualna implementacija zavisit će od vremena i dostupnog NVIDIA GPU-a [2].

3.4. Wald i saradnici (2014): Embree - A Kernel Framework for Efficient CPU Ray Tracing [4]

Embree je važan primjer koliko se ray tracing može optimizovati na CPU-u. Rad opisuje jezgre za brze testove presjeka i obilazak BVH-a, uz oslanjanje na višejezgrene procesore, SIMD instrukcije kao što su SSE i AVX te pažljiv raspored podataka u memoriji. Autori razmatraju obradu pojedinačnih zraka i paketa zraka, jer slični smjerovi često omogućavaju bolje iskorištenje vektorskih instrukcija.

Najkorisnija pouka za naš projekat jeste da paralelizacija nije samo povećavanje broja niti. Nepravilni pristupi memoriji, razgranat obilazak BVH-a i različiti pravci zraka mogu ograničiti ubrzanje. Zbog toga bismo, uz OpenMP, željeli posmatrati i lokalnost podataka, raspodjelu posla i mogućnosti SIMD optimizacije. Embree može poslužiti kao profesionalna referenca za poređenje, ali ne planiramo ga koristiti kao zamjenu za vlastiti osnovni ray tracer [4].

3.5. Wang i saradnici (2022): An Improved Ray Tracing Acceleration Algorithm Based on Bounding Volume Hierarchies [5]

Ovaj rad nastoji poboljšati kvalitet BVH podjela kombinovanjem heuristike površine (SAH) s informacijama o udaljenosti i dometu zraka. Cilj je da se već prilikom organizovanja geometrije smanji vjerovatnoća obilaska nepotrebnih dijelova scene. U eksperimentima autori navode poboljšanje računske efikasnosti od 20 do 35 posto u odnosu na poređeni tradicionalni BVH pristup.

Važno je, ipak, uvažiti područje primjene: istraživanje se odnosi na propagaciju radiotalasa, a ne direktno na klasično grafičko renderovanje. Zato ne možemo očekivati iste rezultate u našem ray traceru. Iz rada preuzimamo koncept SAH-a i činjenicu da način grupisanja trouglova može značajno uticati na broj presjeka. Eventualno poređenje jednostavne podjele i SAH strategije smatraćemo dodatnim dijelom projekta [5].

3.6. Zaključci poređenja naučnih radova

Iako svi radovi polaze od BVH-a, oni zapravo rješavaju različita uska grla. Waldemarson i Doggett [1] ubrzavaju konstrukciju na CPU-u primjenom OpenMP-a; Nishimura [3] istražuje efikasno ažuriranje dinamičnih scena; Meister i Bittner [2] pokazuju bottom-up pristup prilagođen GPU-u. Embree [4] naglašava prednosti SIMD-a i memorijske lokalnosti pri CPU renderovanju, dok Wang i saradnici [5] podsjećaju da je kvalitet prostorne podjele sam po sebi važna optimizacija.

Za naš Checkpoint 1 najvažnije saznanje jeste da treba jasno razlikovati algoritamsko ubrzanje koje donosi BVH od paralelnog ubrzanja koje donose procesorske niti. Projekat zato nećemo zasnivati na reprodukciji svih navedenih radova, nego na njihovim principima: jednostavna i provjerljiva referentna implementacija, jedna ili više razumnih optimizacija te pošteno mjerenje rezultata.

4. Ostala literatura

4.1. Gabriel Gambetta: Computer Graphics from Scratch [6]

Knjiga Gabriela Gambette pristupa računarskoj grafici od osnovnih matematičkih ideja i ne pretpostavlja upotrebu gotovog game enginea. Posebno su korisna poglavlja o generisanju zraka, presjecima s objektima, osvjetljenju, sjenama i refleksijama. Ona nam pomažu da razumijemo šta renderer zapravo računa prije nego što počnemo optimizovati njegov rad. Drugi dio knjige bavi se rasterizacijom, što nije primarni fokus našeg projekta. Gambettinu literaturu koristit ćemo za osnovnu geometriju, kameru i interpretaciju dobivene slike.

4.2. Ray Tracing in One Weekend - serijal i GitHub repozitorij [7]

Serijal Petera Shirleyja, Trevora Davida Blacka i Stevea Hollascha sadrži praktične primjere ray tracinga u C++-u. Prva knjiga objašnjava jednostavan ray tracer: predstavljanje zrake, kameru, presjeke, materijale i zapis slike. Druga knjiga, Ray Tracing: The Next Week, uvodi AABB kutije i BVH stablo te objašnjava kako se može izbjeći provjeravanje svih objekata. Treća knjiga razvija matematičku stranu Monte Carlo uzorkovanja i kvaliteta slike.

Za naš projektni prototip najviše ćemo koristiti prve dvije knjige kao obrazovni oslonac. Ne planiramo samo kopirati primjere: namjera je izdvojiti sekvencijalnu verziju, dodati BVH i vlastiti dio o OpenMP paralelizaciji, mjerenju i analizi performansi. Naprednije vizuelne efekte iz treće knjige nećemo tretirati kao neophodne za Checkpoint 1.

5. Planirani koncept i tehnološki okvir

Planiramo napraviti samostalni C++ program koji će učitati ili definisati jednostavnu 3D scenu, poslati zrake iz kamere i generisati konačnu statičnu sliku. Izrada igre, interaktivnog 3D okruženja ili integracija u Unity/Unreal nije cilj. Bitno je da isti prikaz scene možemo dobiti različitim načinima izvršavanja i uporediti njihov trošak. Prvo ćemo raditi s jednostavnom geometrijom, a za ozbiljnije testove uvesti veći broj trouglova ili 3D model.

Dio sistema	Planirane tehnologije i namjena
Osnovni program	C++17 i CMake za prenosivu, optimizovanu izgradnju projekta.
CPU paralelizacija	OpenMP za podjelu obrade piksela i zraka na više procesorskih niti.
Dodatna optimizacija	SIMD (SSE/AVX na odgovarajućem x86 hardveru), uz praćenje lokalnosti memorije.
GPU opcija	NVIDIA CUDA
Prikaz i analiza	PPM ili PNG slika; CSV podaci i Python grafikoni; Git za zajednički rad.

OpenMP omogućava direktno mjerenje skaliranja s brojem niti, a analiza SIMD-a i memorijske lokalnosti povezuje projekat s načinom na koji procesor zaista izvršava kod. CUDA ostaje mogućnost, ali nije uslov za funkcionalnu CPU verziju.

6. Planirani rezultati i način evaluacije

Završna demonstracija trebala bi prikazati istu scenu kroz tri osnovne varijante: V1 - sekvencijalni ray tracer bez BVH-a, V2 - sekvencijalni ray tracer s BVH-om i V3 - BVH ray tracer paralelizovan OpenMP-om. Te verzije omogućavaju da odvojeno pokažemo efekat boljeg algoritma i efekat korištenja više procesorskih jezgri. Ako vrijeme dozvoli, dodat ćemo poređenje strategija konstrukcije BVH-a ili eksperiment sa SIMD instrukcijama; eventualna CUDA verzija predstavljala bi zaseban korak.

Za poređenje planiramo mjeriti vrijeme konstrukcije BVH-a, vrijeme renderovanja, ukupno vrijeme, broj testova presjeka, broj obrađenih zraka po sekundi i ostvareno ubrzanje. Testovi će koristiti iste scene, rezolucije i postavke renderovanja, uz više ponavljanja na dokumentovanom hardveru. Mijenjat ćemo broj objekata i broj OpenMP niti, a rezultate predstaviti tabelama i grafovima. Slike moraju biti jednake ili vizuelno ekvivalentne, kako optimizacija ne bi bila postignuta na račun ispravnosti prikaza.

6.1. Predviđeni tok rada

Prvi korak bit će izrada male scene i sekvencijalnog renderera koji daje provjerljiv rezultat. Zatim ćemo dodati jednostavnu BVH strukturu i provjeriti da li obje verzije pronalaze iste presjeke. Nakon toga slijedi OpenMP paralelizacija piksela, uz mjerenje rada s različitim brojem niti. Na osnovu rezultata odlučit ćemo ima li smisla proširiti eksperimente na raspodjelu posla, cache lokalnost, SAH ili SIMD. Ovakav redoslijed omogućava da najprije imamo funkcionalan projekat, a zatim dodajemo optimizacije koje možemo objasniti i objektivno ocijeniti.

7. Zaključak

Istraživanje za Checkpoint 1 pokazalo nam je da BVH nije alternativni način renderovanja, nego struktura koja ubrzava pronalaženje presjeka unutar ray tracera. Naučni radovi ukazuju da performanse zavise od načina izgradnje i obilaska stabla, raspodjele posla po nitima i efikasnosti pristupa memoriji. Zato ćemo prvenstveno razviti CPU verzije s OpenMP-om i njihovim mjerljivim poređenjem. Time ćemo na konkretnom problemu moći primijeniti principe paralelnog programiranja koje obrađujemo na predmetu.

8. Literatura

[1] Waldemarson, G., Doggett, M. (2025). Parallel Axis Split Tasks for Bounding Volume Construction with OpenMP. GRAPP/VISIGRAPP. doi:10.5220/0013317100003912.

[2] Meister, D., Bittner, J. (2017/2018). Parallel Locally-Ordered Clustering for Bounding Volume Hierarchy Construction. IEEE TVCG. dcgi.fel.cvut.cz/projects/ploc/ploc-tvcg.pdf.

[3] Nishimura, S. (2021). Grid-induced bounding volume hierarchy for ray tracing dynamic scenes. The Visual Computer. doi:10.1007/s00371-021-02205-3.

[4] Wald, I., Woop, S., Benthin, C., Johnson, G. S., Ernst, M. (2014). Embree: A Kernel Framework for Efficient CPU Ray Tracing. ACM Transactions on Graphics. embree.org/papers/2014-Siggraph-Embree.pdf.

[5] Wang, C. i sar. (2022). An Improved Ray Tracing Acceleration Algorithm Based on Bounding Volume Hierarchies. arXiv:2208.10008.

[6] Gambetta, G. Computer Graphics from Scratch. gabrielgambetta.com/computer-graphics-from-scratch/.

[7] Shirley, P., Black, T. D., Hollasch, S. Ray Tracing in One Weekend (serijal i kod). github.com/RayTracing/raytracing.github.io.